

POO: más allá de Java

Joseline Mazariegos — 261742

Sección 1

Concepto	Java	C#
Declarar una clase	<code>class Persona { }</code>	<code>class Persona { }</code>
Crear un objeto	<code>Persona p = new Persona();</code>	<code>Persona p = new Persona();</code>
Atributos y visibilidad	<code>private String nombre;</code> <code>public int edad;</code>	<code>private string nombre;</code> <code>public int Edad;</code>
Constructor	<code>Persona(String n)</code> <code>{ nombre=n; }</code>	<code>Persona(string n)</code> <code>{ nombre=n; }</code>
Método y llamada	<code>public String saludar(){ return</code> <code>"Hola";}</code> <code>p.saludar();</code>	<code>public string Saludar(){ return</code> <code>"Hola";}</code> <code>p.Saludar();</code>
Polimorfismo	<code>abstract class Figura {</code> <code>abstract double area();</code> <code>}</code>	<code>abstract class Figura {</code> <code>public abstract double Area();</code> <code>}</code>

En el ejemplo de polimorfismo, las clases concretas heredan de Figura y cada una define su propio cálculo del área. En C# se escribe : para indicar herencia y override para redefinir un método virtual o abstracto.

2. ¿CÓMO SE CONTROLA EL ACCESO A LOS ATRIBUTOS EN C#?

C# tiene modificadores como `private`, `public` y `protected`. Un campo `private` solo se puede usar dentro de su clase. Para proteger los datos, se suele dejar el campo privado y permitir el acceso con una propiedad o con métodos que revisen los cambios. Esto se parece a Java, que también permite campos privados y métodos getters y setters. Así se evita que cualquier parte del programa cambie un dato sin respetar las reglas de la clase.

3. ¿C# PERMITE SOBRECARGA DE MÉTODOS?

Sí. C# permite tener varios métodos con el mismo nombre si reciben distintos parámetros. Por ejemplo, se puede tener `Sumar(int a, int b)` y `Sumar(int a, int b, int c)`. Al llamar el método, el compilador elige la versión que coincide con los argumentos. También se pueden usar parámetros opcionales.

Sección 2

El programa usa una clase base abstracta llamada Figura y tres figuras concretas. Valida las medidas en sus constructores para impedir que se creen figuras con dimensiones cero o negativas.

CODIGO EN C#:

```
using System;
using System.Collections.Generic;

namespace FigurasApp
{
    abstract class Figura
    {
        private string nombre;

        protected Figura(string nombre)
        {
            this.nombre = nombre;
        }

        public string ObtenerNombre()
        {
            return nombre;
        }

        public abstract double CalcularArea();

        public string Describir()
        {
            return ObtenerNombre() + " con área " + CalcularArea().ToString("F2");
        }
    }

    class Circulo : Figura
    {
        private double radio;

        public Circulo(double radio) : base("Círculo")
        {
            if (radio <= 0)
            {
                throw new ArgumentException("El radio debe ser positivo.");
            }

            this.radio = radio;
        }

        public override double CalcularArea()
        {
            return Math.PI * radio * radio;
        }
    }

    class Rectangulo : Figura
    {
        private double baseFigura;
        private double altura;
```

```

public Rectangulo(double baseFigura, double altura) : base("Rectángulo")
{
    if (baseFigura <= 0 || altura <= 0)
    {
        throw new ArgumentException("La base y la altura deben ser positivas.");
    }

    this.baseFigura = baseFigura;
    this.altura = altura;
}

public override double CalcularArea()
{
    return baseFigura * altura;
}
}

class Triangulo : Figura
{
    private double baseFigura;
    private double altura;

    public Triangulo(double baseFigura, double altura) : base("Triángulo")
    {
        if (baseFigura <= 0 || altura <= 0)
        {
            throw new ArgumentException("La base y la altura deben ser positivas.");
        }

        this.baseFigura = baseFigura;
        this.altura = altura;
    }

    public override double CalcularArea()
    {
        return baseFigura * altura / 2;
    }
}

class Lenguajes
{
    static void Main(string[] args)
    {
        List<Figura> figuras = new List<Figura>();

        figuras.Add(new Circulo(2));
        figuras.Add(new Rectangulo(3, 4));
        figuras.Add(new Triangulo(3, 4));

        foreach (Figura figura in figuras)
        {
            Console.WriteLine(figura.Describir());
        }
    }
}

```

```
        Console.ReadKey();  
    }  
}
```

SALIDA:

Círculo con área 12.57

Rectángulo con área 12.00

Triángulo con área 6.00

La lista está declarada como `List<Figura>`, pero contiene un círculo, un rectángulo y un triángulo. El ciclo llama a

`Describir()` para todos. Cada objeto usa su propio método `CalcularArea()`; eso demuestra el polimorfismo.

Sección 3

1. DIFERENCIAS EN LA FORMA DE LOGRAR POLIMORFISMO

Java y C# lo hacen de una forma bastante parecida. En los dos se puede crear una clase abstracta y hacer que otras clases hereden de ella. En mi código, `Circulo`, `Rectangulo` y `Triangulo` heredan de `Figura` y cada una redefine `CalcularArea()`. Después se guardan juntas en una lista de `Figura`. Me parece explícito porque el tipo base y los métodos que debe implementar cada clase están escritos claramente. A la vez, permite flexibilidad porque el ciclo funciona con nuevas figuras sin tener que preguntar qué tipo es cada objeto.

2. ¿CÓMO SE MODELA EN GO SIN CLASES TRADICIONALES?

Go no usa clases ni herencia de clases como Java. Usa estructuras para agrupar datos y métodos para definir comportamiento. También tiene interfaces, que describen los métodos que un tipo debe tener. Por ejemplo, una interfaz `Figura` podría pedir el método `CalcularArea()`. Una estructura `Circulo` que tenga ese método puede usarse como `Figura` aunque no declare explícitamente que hereda de ella. Así se pueden tratar distintos tipos de manera común usando interfaces.

3. TIPADO ESTÁTICO EN C# COMPARADO CON JAVASCRIPT Y JAVA

C# y Java tienen tipado estático. En este programa la lista acepta objetos `Figura`, por lo que el compilador puede avisar si intento agregar un objeto que no corresponda o si llamo un método que el tipo no ofrece. Eso ayuda a encontrar algunos errores antes de ejecutar. JavaScript tiene tipado dinámico: se puede guardar en un arreglo un objeto que no tenga `calcularArea()`. El error aparece cuando el programa intenta llamar ese método que no existe. La flexibilidad puede hacer más rápido escribir código, pero algunos errores aparecen hasta la ejecución. El tipado estático revisa más cosas antes, aunque requiere declarar los tipos y seguirlos.

4. ENCAPSULAMIENTO Y EL CASO DE CUENTA

El encapsulamiento busca proteger los datos y controlar cómo se modifican, aunque la forma de hacerlo depende del lenguaje. Si `Cuenta` tuviera saldo público, otra parte del programa podría cambiarlo directamente y asignar, por ejemplo, un saldo negativo sin usar las reglas de la cuenta.

```

class Cuenta
{
private decimal saldo;
public decimal ObtenerSaldo()
{
return saldo;
}
{
public void Depositar(decimal cantidad)
if (cantidad <= 0)
throw new ArgumentException("La cantidad debe ser positiva.");
saldo = saldo + cantidad;
}
}

```

En esta versión el campo saldo es privado. Se puede consultar mediante ObtenerSaldo() y solo se modifica con Depositar(), que verifica que la cantidad sea positiva. De esta manera la clase mantiene el control sobre sus datos.

REFERENCIAS

Oracle. Java Tutorials: Classes and Objects. <https://docs.oracle.com/javase/tutorial/java/javaOO/>

Microsoft Learn. Classes and objects in C#. <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/types/classes>

Microsoft Learn. Access modifiers. <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/access-modifiers>

The Go Authors. A Tour of Go: Interfaces. <https://go.dev/tour/methods/9>